MEMORIA TÉCNICA DEL PROYECTO

DR. AGRAMONTE

Plataforma web de gestión de citas médicas

Autora: Mar Agramonte Campos
En línea: https://www.dragramonte.com

Resumen

Dr. Agramonte es una aplicación web para que pacientes reserven, consulten y cancelen citas médicas sin depender del teléfono del profesional. El sistema integra un frontend estático (HTML, CSS y JavaScript), una API REST con **Spring Boot 3.5.3**, base de datos **PostgreSQL**, autenticación **JWT**, control de concurrencia con **bloqueo pesimista**, notificaciones por dos canales independientes —**Twilio** (SMS/WhatsApp) y un **bot de Telegram**— y despliegue reproducible con **Docker Compose**. La aplicación está además **desplegada en la nube y es accesible públicamente con HTTPS** en `https://www.dragramonte.com`.

El proyecto nace de una observación directa del sector sanitario: buena parte de las llamadas que recibe un médico fuera de su horario no son urgencias clínicas, sino gestión de agenda. Digitalizar esa parte administrativa devuelve tiempo al profesional sin sustituir el trato humano donde de verdad hace falta. Sobre esa idea se construye una herramienta pensada para consultas pequeñas y médicos autónomos, que mantienen la titularidad de sus datos en lugar de depender de plataformas de terceros.

La interfaz replica la identidad visual del consultorio: paleta **teal** (#0F766E, #14B8A6), tipografías DM Sans / DM Serif Display, modo claro/oscuro y diseño responsive. La reserva se organiza en pasos (centro, calendario, hora, datos y confirmación) y puede iniciarse desde `servicios.html` con el tipo de consulta preseleccionado. Un **cuadro de mando** restringido por rol convierte los datos operativos en información de gestión (carga por médico, demanda por especialidad, tasa de cancelación) y la interfaz sigue las pautas de accesibilidad **WCAG 2.1 AA**.

La calidad se respalda con una batería de **pruebas automatizadas** (unitarias, de seguridad y de integración con Testcontainers) y con análisis de vulnerabilidades **OWASP dependency-check**. La principal conclusión es que la coherencia entre memoria, código y demostración es tan determinante como la propia funcionalidad: este documento describe la versión *real* del sistema —la que está desplegada y en funcionamiento—, distinguiendo en todo momento lo entregado de lo planificado como evolución.

Palabras clave: citas médicas, aplicación web, Spring Boot, API REST, PostgreSQL, JWT, concurrencia, Twilio, Telegram, Docker, despliegue en la nube, HTTPS, PWA, accesibilidad.

Abstract

Dr. Agramonte is a web application that lets patients book, review and cancel medical appointments without depending on the practitioner's phone line. The system integrates a static frontend (HTML, CSS and JavaScript), a REST API built with **Spring Boot 3.5.3**, a **PostgreSQL** database, **JWT** authentication, concurrency control through **pessimistic locking**, notifications over two independent channels —**Twilio** (SMS/WhatsApp) and a **Telegram bot**— and reproducible deployment with **Docker Compose**. The application is also **deployed to the cloud and publicly accessible over HTTPS** at `https://www.dragramonte.com`.

The project stems from direct observation of the healthcare sector: many of the calls a doctor receives outside working hours are not clinical emergencies but scheduling tasks. Digitising that administrative layer gives time back to the professional without replacing human contact where it is truly needed. On this premise, the tool is designed for small practices and self-employed doctors, who retain ownership of their data instead of relying on third-party platforms.

The interface mirrors the practice's visual identity: a **teal** palette (#0F766E, #14B8A6), DM Sans / DM Serif Display typefaces, light/dark mode and responsive design. Booking is organised in steps (centre, calendar, time, details and confirmation) and can be started from `servicios.html` with the appointment type preselected. A role-restricted **management dashboard** turns operational data into business insight (workload per doctor, demand per specialty, cancellation rate), and the interface follows **WCAG 2.1 AA** accessibility guidelines.

Quality is backed by a suite of **automated tests** (unit, security and Testcontainers-based integration) and **OWASP dependency-check** vulnerability analysis. The main conclusion is that consistency between report, code and demonstration is as decisive as functionality itself: this document describes the *real* state of the system —the one deployed and running—, consistently distinguishing what has been delivered from what is planned as future work.

Keywords: medical appointments, web application, Spring Boot, REST API, PostgreSQL, JWT, concurrency, Twilio, Docker, cloud deployment, HTTPS, PWA, accessibility.

Índice general

Resumen 1

Abstract 3

1 Justificación y contexto 6
- 1.1 De dónde viene este proyecto . . . 6
- 1.2 La carga administrativa invisible . . . 6
- 1.3 El coste oculto para el profesional . . . 7
- 1.4 El hueco que existe en el mercado . . . 7
- 1.5 El sector: construir software para una consulta médica . . . 8
- 1.6 Tipos de solución y dónde encaja el proyecto . . . 8
- 1.7 Viabilidad económica (estimación) . . . 9
- 1.8 Por qué y para qué . . . 10
- 1.9 Marco legal aplicable . . . 10

2 Objetivos 11
- 2.1 Objetivo general . . . 11
- 2.2 Objetivos específicos . . . 11

3 Desarrollo 13
- 3.1 Del boceto inicial a la implementación . . . 13
- 3.2 Metodología de desarrollo . . . 14
- 3.3 Arquitectura general . . . 14
- 3.4 Organización del backend en capas y DTOs . . . 15
- 3.5 Stack tecnológico . . . 16
- 3.6 Justificación de las tecnologías . . . 16
- 3.7 Modelo de datos . . . 17
- 3.8 Persistencia relacional frente a NoSQL . . . 19
- 3.9 Frontend y paleta visual . . . 20
- 3.10 Autenticación JWT . . . 20
- 3.11 Reserva y concurrencia . . . 21
- 3.12 Reserva como invitado . . . 22
- 3.13 API principal . . . 25

3.14 Manejo centralizado de errores 25
3.15 Notificaciones: Twilio y Telegram 26
3.16 Implementación del frontend 27
3.17 Accesibilidad y usabilidad (WCAG) 30
3.18 Sincronización de citas: una sola fuente de verdad 32
3.19 Cuadro de mando de gestión (analítica) 32
3.20 Seguridad y RGPD 34
3.21 Despliegue 36
3.22 Sostenibilidad y eficiencia (green computing) 37
3.23 Planificación temporal 39
3.24 Pruebas y validación 40

4 Conclusiones **43**

Agradecimientos **46**

Bibliografía **47**

Anexo **49**

Capítulo 1:

Justificación y contexto

1.1 De dónde viene este proyecto

Me he criado rodeada de sanidad. La mayoría de mis familiares y amigos trabajan en el sector, así que lo que sé de esta profesión no viene de un libro: viene de haberla visto de cerca durante años. Y lo que he visto se repite con una regularidad que asusta: un médico no desconecta cuando cierra la consulta. El teléfono sigue sonando. En Navidad, en verano, en mitad de una cena, incluso en mitad de un viaje. Siempre hay alguien que llama, ya sea para confirmar una cita, preguntar por una receta o pedir un consejo que, sinceramente, podría esperar.

La mayoría de esas llamadas no son urgencias. Son gestiones que, con la herramienta adecuada, el paciente podría resolver por su cuenta. Lo que muchas veces se pasa por alto es que ese médico también tiene familia, también necesita estar en una cena sin el teléfono encima y también tiene derecho a un domingo sin interrupciones. La tecnología no lo soluciona todo, pero sí puede encargarse de la parte administrativa que hoy recae directamente sobre el profesional. De ahí nace este proyecto: no es una idea inventada, es algo que observé durante años. Si el paciente puede gestionar sus citas solo, el médico recupera tiempo. Es así de simple.

1.2 La carga administrativa invisible

El problema no es solo cuántas llamadas entran, es quién acaba cogiéndolas. En muchas consultas pequeñas, o en centros que van desbordados, no hay personal de administración, así que el médico termina haciendo también ese trabajo. Cada vez que descuelga el teléfono para confirmar una cita o resolver un trámite, deja a medias lo que estaba haciendo con el paciente que tiene delante.

Este proyecto no quiere sustituir el trato humano ni apartar al profesional

del teléfono. Hay momentos en los que quien llama está asustado, confuso o simplemente necesita que alguien lo escuche, y para eso no vale una aplicación. Lo que busco es que el médico no tenga que coger el teléfono cada vez que alguien llama para confirmar que su cita del jueves sigue siendo a las cinco, ni cada vez que un paciente aprovecha que tiene su número para preguntar una duda que podría esperar a la próxima consulta. Eso sí lo puede hacer un sistema, y liberar ese tiempo es, en el fondo, el objetivo de toda la plataforma.

1.3 El coste oculto para el profesional

Hay algo que casi nunca se dice en voz alta. Cuando el médico sale de la consulta por la tarde, no deja de ser médico: sigue pendiente. Si un paciente le llama a las diez de la noche porque le duele algo, coge el teléfono. Si alguien le escribe un domingo por la mañana, contesta. Y todas esas horas en las que está trabajando, aunque nadie lo llame trabajo, no las cobra: están fuera de su jornada, fuera de su sueldo y dentro de su vida.

Y hay algo más que tampoco suele mirarse: nadie pregunta si el médico está bien, si ha dormido, si ha comido, si lleva semanas arrastrando un cansancio que no se permite parar. Como su oficio es cuidar, damos por hecho que ya se cuida solo, y muchas veces no es así. Lo he visto en gente de mi familia y en amigos del sector: pasan el día entero escuchando lo que le duele a todo el mundo y llegan a casa sin margen para contar lo suyo. Por eso entiendo esta aplicación no solo como un gestor de citas, sino como una pequeña forma de devolver un respiro a alguien que llevaba mucho tiempo sin pedirlo.

1.4 El hueco que existe en el mercado

En España, si trabajas en un hospital grande tienes sistemas de cita online; si formas parte de una clínica privada importante, tienes acceso a plataformas como Doctoralia o Top Doctors. Pero si eres un médico independiente o ejerces en una consulta pequeña, lo más probable es que sigas gestionando las citas por teléfono y agenda de papel. Las soluciones existentes presentan carencias para ese perfil:

- **Comisiones y suscripciones** pensadas para grandes volúmenes, que no encajan con quien atiende a 40–50 pacientes a la semana.
- **Dependencia de terceros:** el médico deja de ser dueño de sus propios datos, que pasan a alojarse en servidores ajenos.

- **Personalización limitada:** no se adaptan con facilidad a una especialidad concreta ni al modo de trabajar de una consulta pequeña.

Existe un espacio entre ambas opciones donde caben muchos profesionales que necesitan una herramienta especializada, accesible y bajo su control. **Dr. Agramonte se diseña pensando precisamente en ese espacio.**

1.5 El sector: construir software para una consulta médica

El ámbito del proyecto es el sanitario privado y, dentro de él, un problema muy concreto: la gestión de agenda, la relación paciente–médico y la comunicación entre ambos.

Construir software para una consulta médica exige un perfil mixto: por un lado, trabajo de oficina propio del desarrollo de software (análisis, programación, pruebas, documentación); por otro, la necesidad de entender la realidad de un entorno clínico, donde los datos que se manejan son sensibles, los usuarios tienen edades y niveles digitales muy distintos, y un error no es un simple *bug*, sino una persona afectada. Comprender ese sector —sus tiempos, sus prioridades y sus límites legales— es tan parte del trabajo como escribir el código, y de hecho condiciona casi todas las decisiones técnicas que aparecen más adelante.

1.6 Tipos de solución y dónde encaja el proyecto

Desde el punto de vista del desarrollo, una herramienta de gestión de citas puede abordarse de tres formas, cada una con sus compromisos:

- **SaaS** (software como servicio): el proveedor aloja la aplicación y el cliente paga una suscripción (Doctoralia, Top Doctors). Cómodo y siempre actualizado, pero con costes recurrentes, comisiones y los datos en manos de un tercero.
- **A medida** (*open source* / propietario): la aplicación se desarrolla y se despliega en la infraestructura del propio profesional. Da control total, sin comisiones y permite personalizar, a cambio de requerir conocimiento técnico para mantenerla.
- **Herramientas genéricas** (agendas o CRM no sanitarios): baratas, pero ajenas a las particularidades de la sanidad (historial clínico, normativa de conservación, roles).

Dr. Agramonte se sitúa en el modelo *a medida*: una solución propia, sin comisiones, con los datos bajo control del consultorio y pensada para ser desplegable y mantenible por un perfil técnico.

1.7 Viabilidad económica (estimación)

Como desarrolladora, una decisión técnica no se sostiene solo por ser elegante: tiene que tener sentido también en coste. Por eso conviene una estimación, con cifras aproximadas y a modo ilustrativo, de lo que aporta la herramienta frente a su coste.

Ahorro. Si la reserva, confirmación y cancelación autónomas evitan en torno a **4 horas semanales** de gestión telefónica, y se valora la hora de consulta privada en unos 50 €, el ahorro ronda los 200 € semanales, del orden de **10.000 € al año** en tiempo recuperado por el profesional.

Coste. El desarrollo es un coste *único* ya asumido, y la operación es deliberadamente barata gracias a las decisiones de arquitectura: un frontend estático sin dependencias, un único backend Java y una base de datos relacional caben de sobra en una VPS modesta (del orden de 10 €/mes). No hay comisiones por cita ni cuotas de plataforma de terceros.

Conclusión. Con estas cifras —que habría que afinar en un caso real— la inversión se amortiza en pocos meses. Más allá del número, la idea de fondo es que las decisiones de desarrollo (sin frameworks innecesarios, contenedores ligeros, una sola base de datos relacional) no solo simplifican el código: también hacen el sistema **sostenible** para una consulta pequeña.

Conviene situar el proyecto frente a las alternativas reales. Las grandes plataformas del sector privado (Doctoralia, Top Doctors) están pensadas para clínicas y hospitales con volúmenes altos: aportan visibilidad y un directorio de pacientes, pero a cambio el profesional depende de un intermediario, asume sus condiciones económicas y deja de ser dueño de sus propios datos. En el otro extremo, las herramientas genéricas de gestión (agendas o CRM no sanitarios) son más baratas, pero no entienden las particularidades de la sanidad: no contemplan historial clínico, no se ajustan a la normativa de conservación de datos de salud ni manejan roles propios de la práctica médica. Dr. Agramonte no compite con ninguno de los dos en su terreno; ocupa el hueco intermedio, ofreciendo a una consulta pequeña una solución propia, simple y conforme a la normativa, sin comisiones por cita ni dependencia de un tercero.

1.8 Por qué y para qué

Por qué: las consultas pequeñas y los médicos autónomos carecen de una herramienta de reserva propia, simple y bajo su control. **Para qué:** ofrecer un canal de autoservicio 24/7 que descargue al profesional de la gestión telefónica y mejore la experiencia del paciente, manteniendo la titularidad de los datos en manos del propio consultorio.

1.9 Marco legal aplicable

Cualquier aplicación que gestione datos de salud en Europa está sujeta a un marco legal exigente, precisamente porque se trata de información sensible:

- **RGPD** (Reglamento (UE) 2016/679): los datos de salud son una categoría especial de datos, con un nivel de protección reforzado.
- **LOPDGDD** (Ley Orgánica 3/2018): la adaptación española del RGPD.
- **Ley 41/2002**, básica reguladora de la autonomía del paciente, que regula la documentación clínica y sus plazos de conservación.

Esto significa que no basta con que la aplicación funcione: tiene que funcionar de forma segura y conforme a la ley. La plataforma incorpora política de privacidad, consentimiento explícito en la reserva y minimización de datos (solo se piden los necesarios para gestionar la cita). Los datos son la intimidad de alguien que ha confiado en el sistema, y tratarlos con cuidado no es un extra: es la base.

Capítulo 2:

Objetivos

Los objetivos se formularon con un criterio que ha guiado todo el proyecto: que cada meta fuera **concreta, verificable y realmente alcanzable** con los recursos disponibles. No se persigue prometer un producto enorme a medias, sino entregar un sistema más pequeño pero completo, coherente y demostrable.

2.1 Objetivo general

Desarrollar una aplicación web de gestión de citas médicas que permita al paciente reservar y cancelar de forma autónoma, reduciendo la dependencia del teléfono y devolviendo al profesional el control de su tiempo, sobre un backend seguro y un despliegue reproducible.

2.2 Objetivos específicos

Cada objetivo específico se corresponde con código real del repositorio y, cuando aplica, con una prueba que lo respalda. Esa trazabilidad entre objetivo, implementación y prueba es deliberada: evita el riesgo de prometer en la memoria lo que no existe en el sistema.

1. Modelar y persistir usuarios, médicos, horarios y citas en **PostgreSQL** con migraciones **Flyway** y perfiles `postgres/mysql`.
2. Implementar API REST con **Spring Boot**, autenticación **JWT** y roles `PACIENTE`, `MEDICO` y `ADMIN` en `SecurityConfig`.
3. Garantizar integridad de reservas con `SELECT FOR UPDATE` sobre horarios disponibles.

4. Construir frontend responsive, PWA, modo oscuro y flujo de reserva accesible (`reserva.js`, `api-client.js`).
5. Enviar notificaciones al médico y al paciente por **Twilio** (SMS/WhatsApp) y por un **bot de Telegram**, con recordatorio programado 24 h antes (`CitaReminderScheduler`).
6. Desplegar con **Docker Compose** (nginx + backend + base de datos).
7. Validar la calidad con pruebas automatizadas y análisis de vulnerabilidades, sobre un entorno reproducible.
8. Ofrecer un **cuadro de mando de gestión** con analítica agregada —carga por médico, demanda por especialidad, reparto por centro y tasa de cancelación— restringido a los roles `MEDICO/ADMIN`.

Estos ocho objetivos se retoman en el capítulo de conclusiones para comprobar, uno a uno, su grado de cumplimiento.

Capítulo 3:

Desarrollo

3.1 Del boceto inicial a la implementación

Antes de escribir una sola línea de código dibujé cómo debía sentirse la plataforma. El boceto inicial (figura 3.1) fijó las decisiones de fondo que luego se mantuvieron: un flujo de reserva guiado paso a paso, una estética sobria propia de un consultorio y una navegación pensada para que una persona mayor o poco habituada a la tecnología no se pierda. Ese boceto no fue un trámite: marcó la diferencia entre construir “una web de citas” y construir *esta* web de citas, con su identidad y su tono.



Figura 3.1: Boceto inicial de la plataforma: mapa de secciones y primeras ideas de interfaz.

Comparando el boceto con el resultado final se aprecia que las ideas centrales sobrevivieron al desarrollo (reserva por pasos, selección de centro, identi-

dad visual del consultorio), mientras que otras se ajustaron a lo que era posible construir con garantías en el tiempo disponible. Documentar esa evolución es parte del valor del proyecto: enseña no solo qué se hizo, sino por qué se decidió así.

3.2 Metodología de desarrollo

El proyecto no se construyó de una vez, sino de forma **iterativa e incremental**: primero una web estática que validaba la idea y la navegación, después el backend y la base de datos, luego la integración real entre frontend y API, y por último las notificaciones, la robustez para la demo y la documentación. Cada iteración partía de algo que ya funcionaba y le añadía una capa, lo que permitió detectar pronto los problemas reales —como la incoherencia entre las citas del navegador y las de la base de datos— en lugar de descubrirlos al final.

Todo el código se gestionó con **control de versiones Git**, lo que aportó un historial de cambios, la posibilidad de volver atrás ante un error y una separación clara entre lo que es código fuente y lo que son secretos o configuración local (estos últimos excluidos del repositorio mediante `.gitignore`). Se trabajó además con la distinción entre **entornos**: una configuración para desarrollo y otra preparada para producción, seleccionables por perfiles, de modo que la misma base de código se comporta de forma adecuada en cada contexto sin tocar el código. Esta forma de trabajar es, en sí misma, una de las competencias que el ciclo busca consolidar.

3.3 Arquitectura general

La solución sigue una arquitectura en capas desplegada con contenedores, de forma que cada pieza tiene una responsabilidad clara y puede sustituirse o escalarse sin arrastrar a las demás. El navegador consume el frontend (HTML, CSS y JavaScript) servido por **nginx**; las peticiones a `/api` se redirigen mediante proxy inverso al backend Java, lo que evita problemas de CORS en producción y permite presentar todo bajo un mismo origen. El backend de **Spring Boot** concentra la lógica de negocio, la seguridad y el acceso a datos sobre **PostgreSQL**, y delega en **Twilio** (servicio externo invocado desde `TwilioMessageService`) el envío de SMS y mensajes de WhatsApp. Esta separación en capas (presentación, API, persistencia y servicios externos) es la que sostiene el resto de decisiones técnicas de la memoria.

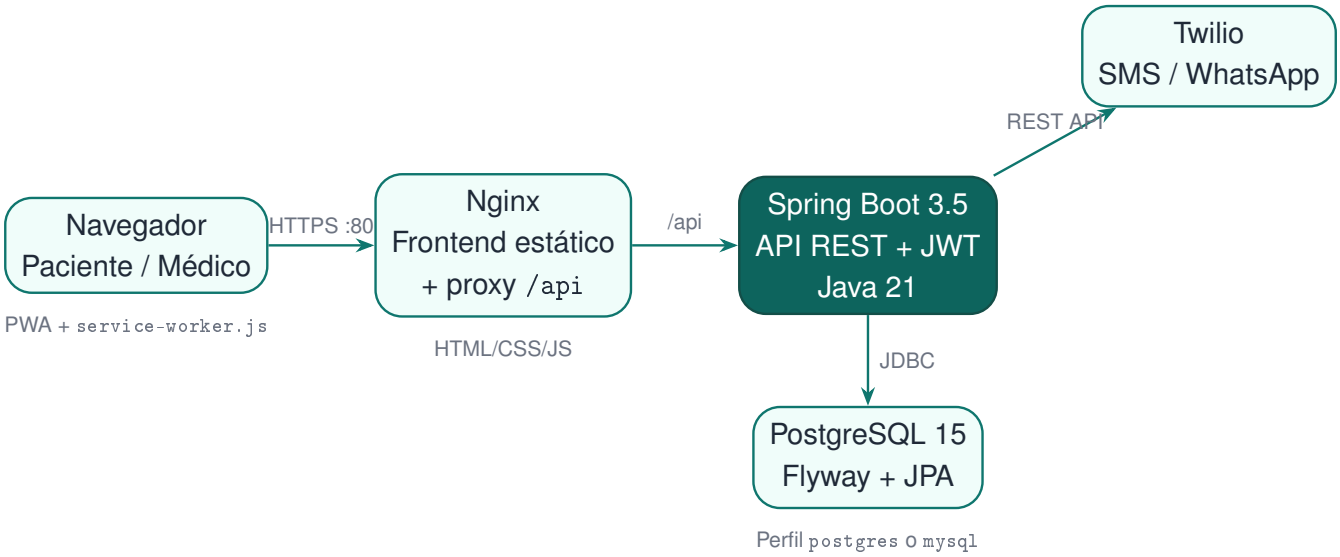


Figura 3.2: Arquitectura desplegada con Docker Compose (implementación actual)

3.4 Organización del backend en capas y DTOs

Dentro del backend, el código sigue el patrón clásico de **tres capas** de Spring, con responsabilidades bien separadas:

- **Controladores** (`controller`): reciben las peticiones HTTP, validan la entrada y devuelven la respuesta. No contienen lógica de negocio; solo orquestan. Ejemplos: `AuthController`, `CitaController`, `MedicoController`, `DisponibilidadController`.
- **Servicios** (`service`): concentran las reglas de negocio (crear una cita, comprobar disponibilidad, decidir si una cancelación es legítima) y abren las transacciones.
- **Repositorios** (`repository`): acceden a la base de datos mediante Spring Data JPA, incluida la consulta con bloqueo pesimista para la reserva.

Una decisión importante fue **no exponer nunca las entidades JPA directamente** en la API. Las respuestas usan objetos de transferencia de datos (*DTO*) como `CitaResponse` o `ReservaPublicaResponse`, y las peticiones se modelan con clases como `CrearCitaRequest` o `ReservaPublicaRequest`. Esto tiene tres ventajas: evita filtrar al exterior campos internos o sensibles, desacopla el modelo de base de datos del contrato público de la API (se puede cambiar la entidad sin romper a los clientes) y permite validar la entrada de forma declarativa con anotaciones. Es una práctica que distingue de inmediato el código profesional del código de juguete.

3.5 Stack tecnológico

Tabla 3.1: Componentes implementados

Capa		
Frontend	HTML5, CSS3, JS ES6+	UI, PWA, `reserva.js`
Proxy / estático	nginx (Docker)	Archivos + proxy `/api`
Backend	Java 21, Spring Boot 3.5.3	API, seguridad, negocio
Persistencia	PostgreSQL 15, Flyway, JPA	Datos transaccionales
Notificaciones	Twilio API (Java SDK)	SMS / WhatsApp
Notificaciones	Bot de Telegram (*webhook* propio)	Segundo canal de avisos, independiente de Twilio
Despliegue	Docker Compose	Entorno reproducible
Analítica	Chart.js 4 (vendorizado)	Cuadro de mando de gestión
Correo	Spring Boot Mail (SMTP)	Enlace de activación de cuenta y formulario de contacto

3.6 Justificación de las tecnologías

Cada pieza del stack se eligió por una razón concreta, no por moda:

- **Java + Spring Boot.** El tipado fuerte de Java detecta en tiempo de compilación errores que en un lenguaje dinámico aparecerían en producción, algo especialmente valioso cuando se manejan datos de salud. Spring Boot aporta, además, un ecosistema maduro: Spring Security como estándar de facto para autenticación y autorización, Spring Data JPA para el acceso a datos y transacciones ACID que garantizan que una reserva se complete entera o no se complete en absoluto.
- **PostgreSQL.** Para datos críticos (usuarios, médicos, horarios y citas) hace falta integridad referencial real, transacciones fiables y bloqueo a nivel de fila. PostgreSQL ofrece las tres cosas y permite implementar el control de concurrencia con `SELECT ...FOR UPDATE`, núcleo de la garantía de “no doble reserva”.

- **Flyway.** Versiona el esquema en migraciones numeradas (V1 a V13), de modo que cualquier entorno —el de desarrollo, el de otra persona del equipo o el de producción— se reconstruye exactamente igual al arrancar.
- **Twilio.** En lugar de montar infraestructura propia de mensajería, se delega el envío de SMS y WhatsApp en un proveedor consolidado, invocado directamente desde el backend Java.
- **Telegram (Bot API).** Se incorporó como segundo canal de aviso por una razón práctica: es gratuito y no exige que el paciente se dé de alta previamente en un *sandbox*, como sí ocurre con la cuenta de prueba de Twilio. Se integra por HTTP con `RestClient`, sin SDK adicional, de modo que no añade ninguna dependencia al proyecto.
- **Docker Compose.** Levanta todo el sistema con un único comando, eliminando el clásico “en mi máquina funciona”.

3.7 Modelo de datos

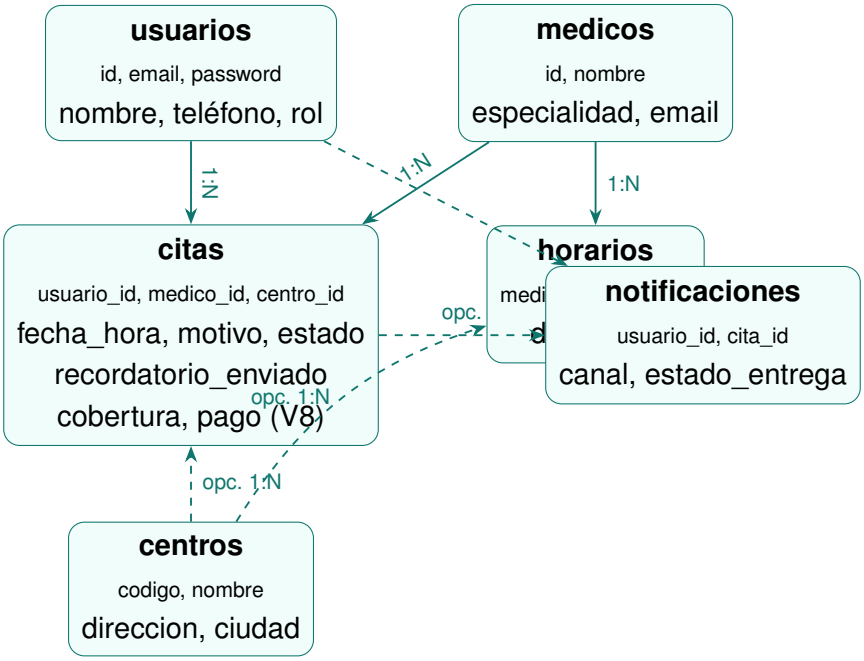


Figura 3.3: Modelo entidad-relación principal (PostgreSQL, migraciones Flyway V1–V13)

El modelo relacional gira en torno a cuatro entidades centrales —`usuarios`, `medicos`, `horarios` y `citas`— más `notificaciones` para el registro de avisos. Un `usuario` (paciente) puede tener muchas `citas`; un `medico` tiene muchos `horarios`

y muchas `citas`; cada `cita` enlaza un usuario con un médico en una fecha y hora concretas. Cada tabla cumple un papel preciso:

- `usuarios`: identidad y credenciales (email único, contraseña cifrada, nombre, teléfono, rol). El campo `rol` discrimina entre `PACIENTE`, `MEDICO` y `ADMIN`, y `cuenta_invitada` marca las cuentas creadas en una reserva sin registro. Desde la V9 guarda además el chat de Telegram vinculado (`telegram_chat_id`) y el token de un solo uso con el que se realiza esa vinculación, y desde la V13 el token con el que el dueño de un buzón convierte su cuenta invitada en una cuenta con contraseña (`activacion_token`, guardado cifrado y con caducidad).
- `medicos`: datos profesionales (nombre, especialidad, email de contacto del facultativo) y, desde la V12, la cuenta desde la que entra ese profesional (`usuario_id`, con índice único). Es lo que permite deducir del JWT de quién es una agenda en lugar de aceptarlo por parámetro.
- `horarios`: la disponibilidad real, como franjas con `inicio`, `fin`, un indicador `disponible` y el centro donde se atiende (`centro_id`, V6). Un índice único sobre `(medico_id, inicio)` impide crear dos franjas idénticas para el mismo médico.
- `citas`: las reservas, que vinculan `usuario_id` y `medico_id` con `fecha_hora`, `motivo` y `estado`. La cancelación no borra la fila: cambia su `estado` (mediante el enumerado `EstadoCita`), preservando la trazabilidad. La V11 añade `acceso_token_hash`, la credencial con la que un invitado consulta y cancela *su* cita sin tener cuenta.
- `notificaciones`: el registro auditable de los avisos enviados, con su canal y estado de entrega.
- `centros`: los centros o consultas donde se atiende (`codigo` único, nombre, dirección, ciudad). Cada `cita` referencia opcionalmente su centro mediante `centro_id`.

La evolución del esquema queda reflejada en las migraciones Flyway: la `V2__cita_recordatorio` añade el campo `recordatorio_enviado` para evitar duplicar avisos Twilio, la `V3` extiende los horarios y crea el usuario médico de prueba, la `V4` introduce `cuenta_invitada` para la reserva sin registro previo, la `V5` crea la tabla `centros` (con sus datos iniciales) y añade `centro_id` a `citas`, la `V6` añade `centro_id` a `horarios` (repartiendo la agenda por centro), la `V7` actualiza los centros a sus ubicaciones reales en Palma de Mallorca, la `V8` incorpora a la cita la cobertura y la preferencia de pago, la `V9` abre el canal de Telegram (`telegram_chat_id` y `telegram_link_token`, ambos con índice único), la

`V10` añade la caducidad de ese token de vinculación, la `V11` introduce el token de acceso a la cita del invitado, la `V12` vincula cada ficha de médico con la cuenta desde la que entra (`medicos.usuario_id`) y la `V13` añade el token de activación de una cuenta invitada. Versionar el esquema de esta forma garantiza que la base de datos se construya exactamente igual en cualquier máquina.

3.8 Persistencia relacional frente a NoSQL

La elección de la base de datos no fue automática: se valoró expresamente un enfoque **NoSQL documental** (MongoDB) frente al **relacional** (PostgreSQL) antes de decidir. Ambos paradigmas son válidos, pero resuelven problemas distintos, y conviene justificar por qué uno encaja mejor en este caso.

Una base de datos documental como MongoDB brilla cuando los datos son heterogéneos y poco estructurados, cuando el esquema cambia con frecuencia y cuando se prioriza la escalabilidad horizontal sobre la consistencia inmediata; guarda documentos JSON flexibles y evita las uniones (*joins*). Una base relacional, en cambio, destaca cuando los datos están fuertemente interrelacionados, cuando la integridad referencial y las transacciones **ACID** son innegociables y cuando hace falta bloqueo a nivel de fila.

En este proyecto los datos son intrínsecamente relacionales —un `usuario` reserva con un `medico` en un `horario` de un `centro`, generando una `cita`— y la invariante crítica del sistema es que **dos pacientes no reserven la misma franja**. Garantizar eso exige una transacción con bloqueo (`SELECT ...FOR UPDATE`), justo donde el modelo relacional es más fuerte y el documental obligaría a soluciones más artificiales. Además, el volumen de una consulta médica es modesto, de modo que la escalabilidad horizontal de NoSQL no aporta una ventaja real aquí. Por todo ello, PostgreSQL es la opción correcta para el *núcleo transaccional*.

Esto no descarta NoSQL: simplemente lo reserva para donde sí encaja. El **historial clínico** es el candidato natural a un almacén documental, porque cada especialidad registra campos distintos (no hay un esquema único razonable) y los datos se añaden más que se modifican; lo mismo aplicaría a un *registro de auditoría* de accesos. La evolución coherente del sistema sería, por tanto, una **persistencia políglota**: PostgreSQL para el núcleo de citas (consistencia) y MongoDB para el historial clínico (flexibilidad), una línea ya recogida entre los trabajos futuros. La decisión, en resumen, no fue “relacional porque sí”, sino elegir la herramienta adecuada para cada tipo de dato.

3.9 Frontend y paleta visual

Los colores del documento y de los diagramas coinciden con `dr-agramonte.css`: teal principal, fondos `off-white`, tarjetas blancas y variante oscura (`data-theme="dark"`, fondo #0F172A).

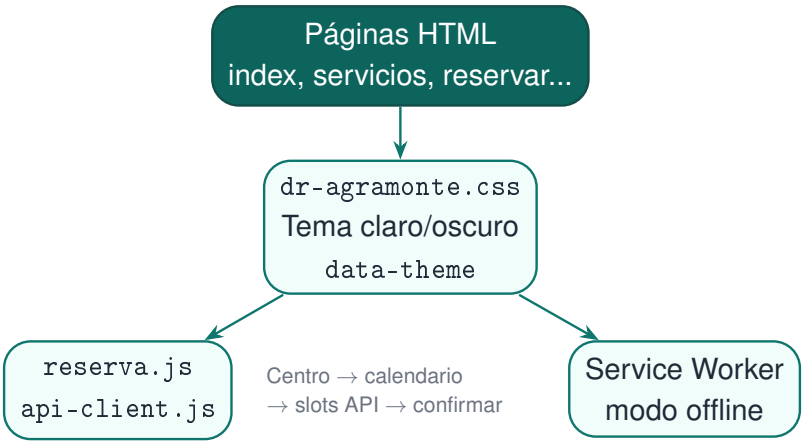


Figura 3.4: Capas del frontend y flujo de reserva en cuatro pasos

3.10 Autenticación JWT

La autenticación se resuelve con **JSON Web Tokens** en modo *stateless*: el servidor no guarda sesiones en base de datos, sino que confía en un token firmado que el cliente presenta en cada petición. `AuthController` expone `POST /api/auth/login` y `POST /api/auth/registro`; al autenticarse correctamente, `JwtProvider` genera un token firmado con clave secreta (algoritmo HMAC-SHA) que incluye el identificador del usuario y una fecha de expiración. A partir de ahí, el token viaja en la cabecera `Authorization: Bearer` y `JwtAuthFilter` lo valida en cada petición protegida antes de que esta llegue al controlador.

Elegí este modelo por tres motivos concretos. Primero, **rendimiento**: al no consultar sesiones en BD, cada validación es una comprobación de firma en memoria. Segundo, **escalabilidad horizontal**: varios contenedores del backend pueden validar el mismo token sin compartir estado entre ellos. Tercero, **simplicidad de integración**: cualquier cliente (la web actual o, en el futuro, una app móvil) consume la API igual. La contrapartida honesta es que un token *stateless* no se puede revocar antes de que caduque; por eso la duración es corta y la introducción de un *refresh token* queda recogida como mejora futura. Las contraseñas nunca se almacenan en claro: se cifran con `BCryptPasswordEncoder` con factor de coste 12, de modo que incluso ante una fuga de la base de datos los hashes resultan costosísimos de revertir.

3.11 Reserva y concurrencia

El problema más interesante de toda la aplicación es aparentemente trivial: ¿qué pasa si dos pacientes intentan reservar la misma franja en el mismo instante? Sin control, ambos verían el hueco como libre, ambos confirmarían y el médico acabaría con dos personas citadas a la misma hora. Esto no es una hipótesis de laboratorio: en una agenda real ocurre justo en las franjas más demandadas. La solución no puede ser "cruzar los dedos", tiene que ser determinista.

La estrategia elegida es el **bloqueo pesimista** a nivel de base de datos. Cuando llega una reserva, el repositorio recupera el horario con un `SELECT ...FOR UPDATE`, que bloquea esa fila concreta hasta que termina la transacción:

```
Horario horario = horarioRepository
  .findByMedicoIdAndInicioAndDisponibleTrueForUpdate(...)
  .orElseThrow(...);
```

Tras validar, se crea la `Cita` y se marca el horario como no disponible. El envío de la notificación (`CitaNotificationService`) se aplaza deliberadamente al *commit* de la transacción (vía `TransactionSynchronizationManager`, fase `afterCommit`): de este modo no se mantiene el bloqueo de la fila durante la llamada de red a Twilio y, si la transacción se revierte, nunca se envía un aviso de una cita que en realidad no llegó a persistirse. El control de concurrencia es **bloqueo pesimista** (`SELECT ...FOR UPDATE` sobre `Horario`): si dos pacientes reservan la misma franja, una operación gana y la otra recibe **409 Conflict**, de forma determinista y sin reintentos. Esta garantía no se afirma sobre el papel: está **verificada bajo concurrencia real** —dos transacciones simultáneas en hilos distintos compitiendo por la misma franja sobre una base PostgreSQL real— como se detalla en la sección de pruebas.

Que haya huecos que reservar. El bloqueo resuelve quién se queda una franja, pero antes alguien tiene que crearla. Durante mucho tiempo las franjas las sembraban las propias migraciones: la `V3` y la `V6` generaban unos noventa días de agenda contados desde el primer arranque. El problema es que **Flyway ejecuta cada migración una sola vez**, de modo que al consumirse esa ventana la agenda se quedaba vacía y la aplicación dejaba de admitir citas. Y lo hacía en silencio: ningún error, ninguna traza en el registro, simplemente un calendario sin días disponibles. Es el tipo de fallo que no aparece en ninguna prueba porque nada está roto; sencillamente se acabó el dato.

La solución es una tarea programada, `GeneradorAgendaScheduler`, que

cada madrugada añade el tramo que falta al final de la ventana, y que se ejecuta además al arrancar por si la aplicación ha estado parada más tiempo del que la ventana cubre. Solo inserta las franjas que todavía no existen —lo garantiza el índice único sobre `(medico_id, inicio)`—, así que ni resucita un hueco ya reservado ni vuelve a marcar como libre uno ocupado. El patrón semanal (las horas de consulta y qué centro atiende cada día) salió del fichero `.sql` y vive ahora en `AgendaProperties`, que es donde se puede ajustar sin tocar una migración ya aplicada.

En la misma línea se cerró un hueco emparentado: una franja vencida seguía en la tabla marcada como disponible, porque nada la retira al pasar su hora. Bastaba una llamada directa a la API para reservar una cita en una fecha ya pasada. Ahora la validación de entrada y el propio servicio rechazan esas fechas, y la consulta de disponibilidad deja de ofrecerlas.

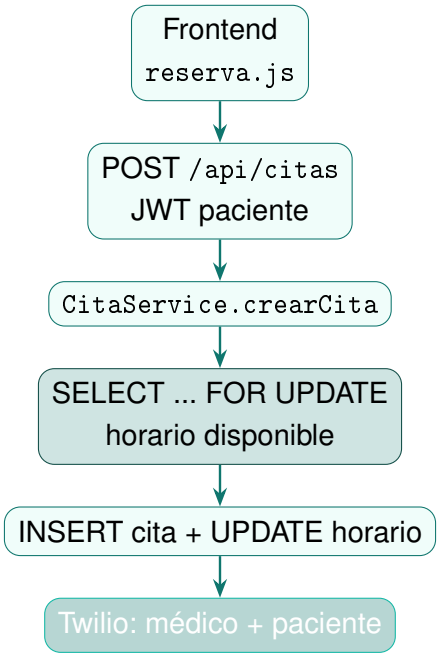


Figura 3.5: Flujo de reserva de cita con bloqueo pesimista y notificaciones

3.12 Reserva como invitado

No todos los pacientes quieren —o saben— crearse una cuenta antes de pedir cita, y obligar a registrarse es una de las principales causas de abandono en cualquier formulario. Por eso la plataforma ofrece **dos vías de reserva que persisten en la misma tabla** `citas`:

- **Paciente autenticado:** `POST /api/citas` con su token JWT.

- **Reserva pública (invitado):** `POST /api/citas/reserva-publica`, sin login previo.

En la reserva de invitado, el backend crea o reutiliza una cuenta de paciente marcada como `cuenta_invitada` (introducida en la migración Flyway V4). Si el email ya pertenece a una cuenta con contraseña, el sistema responde con **HTTP 409** invitando a iniciar sesión, evitando que un tercero reserve en nombre de un usuario registrado.

Todo lo que viene después se apoya en una única idea, que conviene enunciar antes de entrar en el detalle: **un correo electrónico no prueba identidad**. Cualquiera puede escribir la dirección de otra persona en el formulario de reserva, así que el email sirve para *avisar* a un paciente, nunca para *autorizar* a quien dice tenerlo. Las dos piezas siguientes son consecuencia directa de esa regla.

Acceder a la cita sin cuenta. La primera versión identificaba al invitado por su correo: `GET /api/citas/por-email` devolvía sus citas a quien supiera la dirección. Aquello exponía el motivo de consulta en texto libre, que es un dato de salud, a cualquiera que conociera un email. Esa ruta se retiró y en su lugar la reserva emite un **token de acceso a esa cita concreta**: treinta y dos bytes de `SecureRandom` que se entregan una sola vez, en la respuesta de la reserva, y de los que en base de datos solo queda el `SHA-256` (migración **V11**). El token viaja después en la cabecera `X-Cita-Token`, nunca en la URL, porque las rutas quedan registradas en el servidor, en el historial del navegador y en la cabecera `Referer`, y ahí no debe haber una credencial. Cuando el token no corresponde, la respuesta es **404** y no 403: un 403 confirmaría que esa cita existe, y los identificadores son enteros correlativos triviales de recorrer.

Convertir la cuenta de invitado en una cuenta con contraseña. Durante un tiempo el invitado que quería registrarse más tarde se encontraba con un 409 “el email ya está registrado” y se quedaba fuera de su propia cuenta, sin ninguna vía de recuperación. Dejarle poner contraseña sin más tampoco era aceptable: poner contraseña es tomar el control de la cuenta, y con ella se heredan las citas y los motivos de consulta que ya contenga. La única prueba admisible es que el solicitante abra ese buzón, de modo que el alta pasa a tener dos pasos:

1. `POST /api/auth/registro` con el correo de una cuenta invitada responde **202** con un mensaje genérico y **no modifica nada**: ni la contraseña, ni el nombre, ni el teléfono. Lo único que ocurre es que se emite un token de activación y sale un correo con el enlace.

2. `POST /api/auth/activar` cambia ese token por la contraseña elegida, retira la marca de cuenta invitada y devuelve sesión.

El token de activación se genera igual que el de la cita y, como aquel, se persiste solo cifrado con `SHA-256` (migración **V13**): un volcado de la tabla `usuarios` no debe bastar para apropiarse de cuentas ajenas. Es de un solo uso y caduca a los treinta minutos. Un token inexistente, uno caducado y uno ya consumido devuelven **el mismo 400 con el mismo texto**, porque distinguirlos le diría a quien los prueba al azar cuándo ha acertado con uno real. El enlace lleva el token en el fragmento de la URL (detrás de `#`), que el navegador no envía al servidor, así que tampoco queda en los registros de acceso del proxy inverso.

La misma regla gobierna el caso en el que alguien reserva como invitado *enviando* una contraseña sobre un correo que ya tenía cuenta invitada: la cita se crea con normalidad, pero la contraseña no se guarda y lo que sale es el mismo enlace de activación. Y si el servicio de correo no está configurado, estas dos rutas responden **503**: sin correo no hay forma de comprobar quién es el dueño del buzón, y activar la cuenta de todos modos sería exactamente el agujero que el mecanismo viene a cerrar. La consecuencia operativa es que el envío de correo deja de ser opcional para esta funcionalidad, aunque el resto de la plataforma siga funcionando sin él.

Esa misma infraestructura de correo resuelve la limitación que el token de acceso arrastraba: al entregarse una sola vez, la cita quedaba accesible únicamente desde el navegador donde se había reservado, y en la base de datos solo consta su SHA-256, de modo que nadie podía reenviarlo. Ahora la reserva sin cuenta envía también un correo con el enlace de la cita, que lleva el identificador y el token en el *fragmento* de la URL —la parte que el navegador no manda al servidor, por lo que no aparece en los registros de acceso ni en la cabecera `Referer`—; al abrirlo, la página de reserva importa el token en ese dispositivo y lo borra de la barra de direcciones. Los dos correos del sistema siguen reglas deliberadamente opuestas: el de activación *es* la prueba de identidad y sin servicio de correo la operación se niega con un 503, mientras que el de acceso a la cita es una comodidad y se envía tras el *commit* con el fallo capturado, porque un servidor de correo caído no debe impedir reservar una cita ni deshacer una ya confirmada.

Cobertura y preferencia de pago. A raíz del *feedback* de los primeros usuarios de prueba —que echaban en falta saber el precio y los seguros aceptados antes de confirmar— el formulario de reserva captura también la **cobertura**: consulta privada, con su precio visible (80 euros), o seguro médico, con

la aseguradora y el número de tarjeta sanitaria. En el caso privado se recoge además la **preferencia de pago** (en consulta u online). Estos datos se persisten en la cita mediante la migración Flyway **V8** (`cobertura`, `aseguradora`, `numero_tarjeta_sanitaria`, `preferencia_pago`) y se devuelven al profesional al gestionar la agenda. El *cobro* en sí no se procesa todavía: la pasarela de pago en línea se documenta como línea futura.

3.13 API principal

Método	Ruta	Descripción
POST	/api/auth/login, /registro	Sesión JWT; sobre una cuenta invitada el registro responde 202 y envía un enlace
POST	/api/auth/activar	Fija la contraseña de una cuenta invitada con el token del correo
GET	/api/medicos	Listado de especialistas
GET	/api/centros	Listado de centros / consultas
GET	/api/disponibilidad	Slots por médico, fecha y centro
POST	/api/citas	Crear cita (paciente autenticado)
POST	/api/citas/reserva-publica	Reserva sin login (cuenta invitada)
GET	/api/citas/mias	Citas del paciente (JWT)
GET, DELETE	/api/citas/publica/{id}	Consulta y cancelación de la cita de un invitado, con la cabecera `X-Cita-Token`
DELETE	/api/citas/{id}	Cancelación (paciente autenticado)
GET	/api/estadisticas	Cuadro de mando de gestión (rol MEDICO/ADMIN)
GET, POST, DELETE	/api/telegram/vinculacion	Estado, alta y baja del canal Telegram (JWT)
POST	/api/telegram/webhook	Lo invoca Telegram; se autentica con un secreto compartido
POST	/api/contact	Formulario de contacto

Tabla 3.2: Endpoints principales

3.14 Manejo centralizado de errores

Una API es tan buena como sus errores: si cada fallo se devuelve de una forma distinta, el frontend se vuelve frágil y difícil de mantener. Para evitarlo, el backend usa un `GlobalExceptionHandler` que captura las excepciones de forma

centralizada y las traduce a respuestas JSON consistentes, con su código HTTP adecuado. Así, una franja ya ocupada devuelve siempre **409 Conflict**, una acción sin permiso **403 Forbidden** y un recurso inexistente **404 Not Found**, todos con el mismo formato (marca de tiempo, estado, mensaje y ruta).

La ventaja es doble. Por un lado, el frontend puede confiar en una estructura de error predecible y mostrar mensajes claros al usuario (por ejemplo, "esa hora acaba de ocuparse, elige otra" ante un 409). Por otro, se evita filtrar trazas internas o detalles de implementación al exterior, lo que también es una cuestión de seguridad. Centralizar el manejo de errores en un único punto, en lugar de repetir bloques `try-catch` por todos los controladores, mantiene el código limpio y coherente.

3.15 Notificaciones: Twilio y Telegram

Las notificaciones son la otra mitad de la promesa del proyecto: si la web evita llamadas, tiene que avisar de forma fiable de lo que ocurre con cada cita. El envío se realiza **directamente desde el backend Java** mediante el SDK de Twilio, sin un servicio intermedio de colas. Es una arquitectura más simple y perfectamente adecuada al volumen de una consulta; un sistema de mensajería asíncrona con reintentos desacoplados sería justificable a mayor escala y se contempla como evolución futura. `CitaNotificationService` envía:

- Aviso al médico (`TWILIO_NOTIFY_TO`) en alta y cancelación.
- Confirmación y cancelación al paciente, por cada canal que tenga disponible.
- Recordatorio $\sim$24 h antes vía `@Scheduled` (ventana 23–25 h).

Variables: `TWILIO_ENABLED`, `TWILIO_ACCOUNT_SID`, `TWILIO_AUTH_TOKEN`, `TWILIO_FROM`, `TWILIO_CHANNEL` (whatsapp o sms).

El segundo canal: un bot de Telegram. La cuenta de prueba de Twilio impone una limitación incómoda para un producto real: el paciente debe darse de alta en el *sandbox* antes de poder recibir nada, y los mensajes salientes tienen coste. Por eso se añadió un **segundo canal independiente** sobre la *Bot API* de Telegram, que es gratuita y no exige alta previa. La integración es deliberadamente ligera —una llamada HTTP con `RestClient` a `sendMessage`, sin SDK ni dependencias nuevas— y vive en `TelegramMessageService`, gemelo de `TwilioMessageService`.

El punto interesante no es el envío, sino **cómo se averigua a qué chat escribir**. Telegram no permite iniciar una conversación con alguien: es el usuario quien tiene que escribir primero al bot. El flujo implementado resuelve ese problema con un enlace profundo y un token de un solo uso:

1. El paciente, con su sesión iniciada, pulsa *Recibir avisos por Telegram*. El backend (`TelegramVinculacionService`) genera un token de 32 bytes con `SecureRandom`, lo guarda en su fila de `usuarios` con una caducidad de 15 minutos y devuelve el enlace `https://t.me/<bot>?start=<token>`.
2. Al abrirlo, Telegram envía al bot el mensaje `/start <token>`, que llega al backend a través del *webhook* `POST /api/telegram/webhook`.
3. El controlador busca al dueño de ese token, comprueba que no haya caducado, asocia el `chat_id` a su cuenta y **consume el token** (lo pone a `NULL`). A partir de ese momento, ese paciente recibe por Telegram los mismos avisos que por SMS.

Hay dos decisiones de seguridad que conviene subrayar. La primera: el token **se emite siempre para el usuario del JWT**, nunca para un correo recibido por parámetro; de lo contrario cualquiera podría solicitar el token de otra persona y desviar a su propio chat los avisos de las citas ajenas. La segunda: como la ruta del *webhook* es pública —Telegram no puede presentar un JWT—, se autentica con el secreto compartido que se registra en `setWebhook` y que Telegram devuelve en la cabecera `X-Telegram-Bot-Api-Secret-Token`, comparado en tiempo constante. Superada esa verificación, el *webhook* responde siempre `200 OK` aunque el contenido no sirva, porque un error haría que Telegram reintentase el mismo mensaje indefinidamente.

Ambos canales son **independientes y opcionales**: se activan por variables de entorno (`TELEGRAM_ENABLED`, `TELEGRAM_BOT_TOKEN`, `TELEGRAM_BOT_USERNAME`, `TELEGRAM_WEBHOOK_SECRET`) y ninguno de los dos puede tumbar una reserva: los servicios de mensajería nunca propagan la excepción hacia el flujo de citas, solo registran el fallo y devuelven `false`. Si un canal está caído o apagado, el otro sigue avisando. El paciente puede además retirar el canal en cualquier momento (`DELETE /api/telegram/vinculacion`), lo que borra su `chat_id`: una forma concreta de aplicar el derecho de supresión del RGPD.

3.16 Implementación del frontend

El frontend se construyó con HTML5, CSS3 y JavaScript nativo, sin frameworks como React o Vue. No fue la primera idea: al principio valoré usar React

o incluso Python en el cliente, en parte por mi formación y por el *feedback* de profesionales del sector (OmniAccess). Sin embargo, tras un análisis de necesidades reales del proyecto, decidí prescindir de esa maquinaria. Fue una decisión consciente de ingeniería: la aplicación no maneja un estado global tan complejo como para justificarla, y evitarla reduce dependencias, acelera la carga inicial y permite que cualquier persona lea y modifique el código sin aprender antes una librería ni arrastrar un `node_modules` de cientos de megas. El JavaScript se organiza en módulos reutilizables (`api-client.js` para las llamadas REST con JWT, y la lógica de reserva en `reserva.js`), lo que evita duplicar código entre páginas. Saber *cuándo no* añadir una tecnología es, para un perfil de desarrollo, tan importante como saber usarla.

- **Reserva:** `reservar.html` + `reserva.js` (centros, calendario, franjas desde la API y *fallback* local).
- **Servicios:** los enlaces `reservar.html?servicio=...` preconfiguran tipo de consulta y motivo.
- **Tema:** botón luna/sol con preferencia recordada, y clase `btn-reserva` unificada en todo el sitio.
- **PWA:** `manifest.json` y `service-worker.js`; página `offline.html` para cuando no hay conexión.

Construir HTML con datos de otra persona. Prescindir de un framework tiene una contrapartida que conviene nombrar: nadie escapa el HTML por ti. Los paneles se pintaban concatenando plantillas literales de JavaScript y asignándolas a `innerHTML`: dentro de la celda de la tabla se interpolaba directamente el nombre del paciente, tal y como venía del servidor, y ese nombre lo escribe el propio paciente al reservar. Dicho de otro modo: quien reservaba decidía parte del HTML que se pinta en la pantalla del médico, junto con su correo y el motivo de consulta.

La cabecera `Content-Security-Policy` que sirve el proxy inverso impide que un manejador en línea llegue a ejecutarse, porque `script-src 'self'` no los admite, pero sigue permitiendo colar maquetado y enlaces, y la versión de demostración alojada como sitio estático no puede servir esa cabecera. La CSP es la segunda barrera, no la primera. La primera es escapar, y para que escapar sea el camino cómodo se introdujo un pequeño módulo, `js/modules/html.js`, con una plantilla etiquetada que escapa automáticamente todo lo interpolado salvo lo que ya provenga de ella misma. Anidar plantillas o interpolar listas fun-

ciona sin pensar, y olvidarse de escapar deja de ser posible: el camino corto y el seguro pasan a ser el mismo.

Aplicación web progresiva (PWA). La web es instalable en el móvil como si fuera una app, sin pasar por una tienda de aplicaciones. Esto se logra con tres piezas: un `manifest.json` que define nombre, iconos y color de tema; un *service worker* que cachea los recursos críticos para que la navegación siga funcionando aunque la conexión falle, mostrando una página `offline.html` cuando no hay red; y el requisito de servir bajo HTTPS en producción. Para un paciente, la diferencia práctica es poder tener "Dr. Agramonte" como un icono más en su pantalla de inicio, reforzando justamente la idea de un canal propio y cercano al consultorio.

Selección de centro (multi-ubicación). El flujo de reserva permite elegir entre dos centros antes de fijar la hora: *Consulta General Riera* y *Consulta Avenidas*, ambos en Palma de Mallorca y cada uno con sus propios días de atención. Esto refleja una realidad habitual: un mismo profesional atiende en más de una ubicación. El centro **se persiste en la base de datos**: existe una entidad `Centro` (tabla `centros`, creada en la migración Flyway V5) con un `codigo` estable —`madrid`, `palma`— que coincide con el identificador que ya empleaba el frontend, de modo que no hubo que reescribir la web. El backend expone el listado mediante `GET /api/centros`.

Agenda del médico por centro. La multi-ubicación se llevó hasta el final: cada `horario` pertenece a un centro (columna `centro_id`, migración V6; la V7 actualizó los centros a sus ubicaciones reales en Palma de Mallorca), repartiendo la agenda del médico por día de la semana —la consulta de General Riera los lunes, martes y jueves; la de Avenidas los miércoles, viernes y sábados— de forma que cada día el profesional está en un único sitio. A partir de ahí, todo encaja de manera coherente:

- La **disponibilidad se filtra por centro**: `GET /api/disponibilidad?medicoId=&fecha=&cent` solo devuelve los huecos de ese centro (un miércoles no ofrece horas de General Riera).
- La **cita hereda el centro de su horario**: al reservar, el centro no se deduce de lo que diga el cliente, sino de la franja realmente reservada, lo que evita incoherencias.
- La **agenda del médico muestra el centro** de cada cita y puede filtrarse por ubicación.

Todas las relaciones con `centro` son *nullable* a propósito: las citas y horarios anteriores a esta funcionalidad siguen siendo válidos.

Reserva guiada por pasos. La experiencia se divide en pasos pequeños (centro → calendario → hora → datos → confirmación) para reducir la carga cognitiva y no abrumar al usuario con todos los campos a la vez. Esta decisión, heredada del boceto inicial, es especialmente importante para pacientes mayores o poco habituados a la tecnología.

Accesibilidad. La interfaz cuida el uso de etiquetas HTML semánticas, atributos ARIA, un orden de tabulación lógico para navegar solo con teclado y un contraste de color suficiente entre texto y fondo, tanto en modo claro como oscuro. La accesibilidad no se entendió como un requisito formal, sino como la diferencia entre que una persona mayor pueda usar la aplicación o no. Por su importancia en un contexto sanitario, se detalla en su propia sección a continuación.

Validación de datos. La entrada del usuario se valida en dos niveles, porque ninguno de los dos basta por sí solo. En el **frontend** se comprueban formato de email, teléfono español y campos obligatorios antes de enviar, dando un aviso inmediato y evitando viajes innecesarios al servidor; esto mejora la experiencia, pero no es una garantía de seguridad porque puede manipularse. Por eso la validación se repite en el **backend** con las anotaciones de *Bean Validation* sobre los DTO de entrada: el servidor nunca confía en que el cliente haya validado bien. Esta doble barrera —comodidad en el cliente, seguridad en el servidor— es un principio básico de cualquier aplicación web seria.

3.17 Accesibilidad y usabilidad (WCAG)

En una aplicación de salud, la accesibilidad no es un extra: una parte importante de los pacientes son personas mayores o con dificultades visuales o motrices, y si no pueden usar la herramienta, esta sencillamente no cumple su función. Por eso la interfaz se diseñó tomando como referencia las **Pautas de Accesibilidad para el Contenido Web (WCAG 2.1)** en su nivel **AA**, organizadas en torno a sus cuatro principios: perceptible, operable, comprensible y robusto.

Perceptible: contraste de color verificado. El contraste entre texto y fondo se comprobó con la fórmula de luminancia relativa de la WCAG, no a ojo. El texto principal y los elementos interactivos superan con holgura el umbral AA de 4,5:1

para texto normal. Durante la revisión se detectó que el gris más claro usado para textos secundarios se quedaba en 2,5:1 (no cumplía) y se oscureció hasta cumplir, manteniendo en el modo oscuro un tono claro que conserva el contraste sobre fondo oscuro. La tabla siguiente recoge las medidas reales tras el ajuste:

Tabla 3.3: Ratios de contraste medidos (modo claro, sobre fondo blanco)

Uso	Ratio	WCAG AA
Texto principal (gris oscuro)	14,7:1	✓
Botones (blanco sobre teal)	5,5:1	✓
Enlaces y acentos (teal)	5,5:1	✓
Texto secundario (gris medio)	4,8:1	✓
Texto auxiliar / hints (ajustado)	4,6:1	✓
Mensajes de error (rojo)	4,8:1	✓

Operable: teclado y navegación. Toda la aplicación se puede manejar solo con teclado, con un orden de tabulación lógico y un **foco visible** reforzado (un contorno claro con `:focus-visible` de 2 px sobre cada elemento al recibir el foco). Se añadió además un **enlace para saltar al contenido** (“Ir al contenido principal”), oculto hasta que recibe el foco, que permite a quien navega con teclado o lector de pantalla evitar la cabecera repetida en cada página y llegar directamente al contenido; esto cubre el criterio WCAG 2.4.1 (*Evitar bloques*). El flujo de reserva se divide en pasos cortos para exigir los mínimos clics posibles y no abrumar.

Comprensible: estructura y ayuda. El marcado es HTML semántico (encabezados jerárquicos, `nav`, `main`, `footer`), con el idioma declarado (`lang=.es"`) para que los lectores de pantalla lo pronuncien correctamente, atributos ARIA donde el componente lo requiere y una etiqueta asociada a cada campo de formulario. La validación de los datos es inmediata y con mensajes claros, y la reserva guiada por pasos reduce la carga cognitiva, algo especialmente valioso para usuarios poco habituados a la tecnología.

Robusto: preferencias del usuario y zoom. La maquetación usa unidades relativas (`rem`, `clamp`) que permiten ampliar el tamaño de letra del navegador sin que la interfaz se rompa, y respeta la preferencia del sistema `prefers-reduced-motion`, desactivando las animaciones para quienes sufren mareo con el movimiento (problemas vestibulares). El modo claro/oscuro, además de comodidad, ofrece una alternativa a personas sensibles al brillo.

Limitación honesta. La accesibilidad se ha trabajado por diseño y se ha verificado el contraste de forma manual, pero no se ha pasado todavía una auditoría automática completa (por ejemplo, *Lighthouse* o *axe*) ni se ha probado con usuarios reales de lector de pantalla. Documentar esa auditoría y realizar pruebas con usuarios es la evolución natural de este apartado, ya recogida entre las líneas futuras de calidad.

3.18 Sincronización de citas: una sola fuente de verdad

Uno de los problemas más instructivos del desarrollo no fue de algoritmos, sino de *coherencia de datos* entre vistas. Al principio, la lista "Mis citas programadas" que veía el paciente vivía solo en el `localStorage` del navegador, mientras que el panel de pruebas del médico leía de la base de datos. El resultado eran incoherencias desconcertantes: citas que aparecían para el paciente pero no en el panel, cancelaciones que solo existían en un navegador, e incluso reservas antiguas con identificadores locales que nunca habían llegado al servidor.

La solución fue establecer una **única fuente de verdad**: PostgreSQL. La función `syncMisCitas()` de `reserva.js` sustituye la caché local por lo que realmente hay en el servidor, consultando `GET /api/citas/mias` para el paciente autenticado o, para el invitado, recorriendo cita a cita los tokens de acceso que guarda el navegador (`GET /api/citas/publica/{id}` con la cabecera `X-Cita-Token`). Esta sincronización se dispara en los momentos clave: al cargar la página de reserva, tras confirmar o cancelar una cita y tras iniciar o cerrar sesión. El disparador que resincronizaba al teclear el email desapareció con la ruta que lo sostenía: ya no hay nada que un correo por sí solo permita consultar. Así, lo que ve el paciente y lo que ve el panel siempre coinciden, porque ambos derivan del mismo origen. La regla de diseño es sencilla: *el navegador nunca es la verdad; solo refleja la base de datos tras cada sincronización.*

3.19 Cuadro de mando de gestión (analítica)

Una consulta médica no es solo una agenda: es una pequeña empresa que genera datos cada día. Esos datos, si se quedan dispersos en filas de una tabla, no le dicen nada al profesional; pero agregados y representados visualmente revelan información útil para tomar decisiones: qué especialidad se demanda más, cómo se reparte la carga de trabajo, qué centro concentra más actividad o cuántas citas se cancelan. Con esa idea —entender la clínica como un negocio que se puede medir y optimizar— se incorporó un **cuadro de mando de gestión**, accesible solo para el personal interno.

La pieza clave es que toda la analítica se calcula **en la base de datos**, no en memoria. El endpoint `GET /api/estadisticas` se apoya en consultas de agregación `GROUP BY` (con `COUNT`, `JOIN`, `LEFT JOIN`, `COALESCE` para los valores nulos y las funciones de fecha `YEAR()`/`MONTH()`) que devuelven directamente los totales ya calculados. Para no traer las entidades completas a memoria, cada consulta usa una *proyección* ligera de Spring Data —una interfaz con solo los campos necesarios (etiqueta y total)—, lo que mantiene la respuesta pequeña y la consulta eficiente. El servicio que orquesta estas consultas se marca como `@Transactional(readOnly = true)`, una señal explícita de que solo lee datos y nunca los modifica. El cálculo de indicadores derivados, como la tasa de cancelación, se hace en el servicio a partir del reparto por estado, evitando consultas redundantes.

El cuadro de mando presenta primero cuatro indicadores resumen (KPIs) y, debajo, cinco gráficos:

- **Indicadores:** total de citas, citas activas (pendientes y confirmadas), completadas y **tasa de cancelación** como aproximación al ausentismo.
- **Carga de trabajo por médico** (barras): cuántas citas gestiona cada facultativo, de mayor a menor.
- **Demanda por especialidad** (barras): las áreas más solicitadas.
- **Distribución por centro** (barras): el reparto de actividad entre ubicaciones.
- **Reparto por estado** (anillo): la proporción de confirmadas, pendientes, completadas y canceladas.
- **Evolución temporal** (línea): el volumen de citas mes a mes, para detectar tendencias.

En el frontend, los gráficos se dibujan con **Chart.js**, una librería ligera que se incorporó *vendorizada* en el propio repositorio (`js/vendor/`) en lugar de cargarse desde una CDN externa. La decisión es deliberada: el sistema no depende de una conexión a internet ajena para funcionar —coherente con la filosofía PWA y *offline first* del proyecto— y la aplicación se puede mostrar aunque no haya red. La página respeta el modo claro/oscuro recalculando los colores al cambiar de tema, y **degrada con elegancia**: si quien la abre no ha iniciado sesión como personal autorizado, en lugar de un error muestra un aviso claro invitando a identificarse.

El acceso está restringido en `SecurityConfig` a los roles `MEDICO` y `ADMIN`: las analíticas de negocio son una herramienta interna y no deben ser visibles

para un paciente ni para un usuario anónimo. Esta restricción se verificó en vivo —una petición sin token recibe **403 Forbidden**, mientras que con el token de un médico el endpoint devuelve los datos agregados—, lo que conecta este módulo directamente con la política de autorización por rol descrita más adelante. En conjunto, el cuadro de mando demuestra la competencia de *sistemas de gestión empresarial*: convertir los datos operativos del día a día en información de gestión accionable.

3.20 Seguridad y RGPD

La seguridad no se añadió al final, se diseñó desde el principio, porque manejar datos de salud obliga a ello. El acceso está gobernado por roles (`PACIENTE`, `MEDICO`, `ADMIN`) declarados en `SecurityConfig`: cada ruta exige el rol mínimo necesario, de modo que un paciente no puede asomarse a la agenda del médico ni un usuario anónimo crear citas en nombre de otro. Sobre esa base se aplican las medidas concretas de protección de datos:

- Contraseñas con `BCryptPasswordEncoder(12)`.
- CORS configurado para orígenes de desarrollo y producción.
- Política de privacidad en `privacidad.html` y checkbox de aceptación en el formulario de reserva.
- Secretos (`JWT_SECRET`, Twilio, token del bot de Telegram) en `.env`, no en el repositorio.
- **Webhook público autenticado:** la ruta que invoca Telegram no puede presentar un JWT, así que se valida con el secreto compartido que se registra en `setWebhook` y que Telegram reenvía en cada llamada; la comparación se hace en **tiempo constante** (`MessageDigest.isEqual`) para no filtrar información por el tiempo de respuesta.
- **Límite de peticiones por IP:** las rutas públicas que escriben o consumen CPU —inicio de sesión, alta, activación, formulario de contacto y reserva sin cuenta— están limitadas a diez peticiones cada cinco minutos por dirección, y responden **429** con `Retry-After` al excederlo.
- **Datos personales fuera del registro:** el formulario de contacto volcaba al *log* el nombre, el correo, el teléfono y el texto íntegro del mensaje. Hoy solo deja constancia del motivo —una opción de una lista— y de la longitud del mensaje; el contenido viaja por correo, que es su sitio.

- **El correo no autoriza, solo avisa:** ninguna operación que otorgue control sobre una cuenta acepta un email como prueba. Poner contraseña a una cuenta invitada exige el token enviado a ese buzón, y consultar la cita de un invitado exige el token emitido al reservarla. Los dos son secretos de un solo uso y en base de datos solo se guarda su `SHA-256`.
- **Análisis de vulnerabilidades:** `OWASP dependency-check` configurado en `pom.xml`, que hace fallar el build con CVSS ≥ 7.

Estar autenticado no es estar autorizado. Las rutas de la agenda del médico (`/api/citas/agenda/**`) comprobaban el rol correctamente: hacía falta ser `MEDICO` o `ADMIN`. Lo que nadie comprobaba era *qué* médico, porque el profesional llegaba como parámetro de la URL. Bastaba cambiar un número para leer la agenda de otro, con los nombres, los correos, los teléfonos y los motivos de consulta de sus pacientes. Es un fallo que ninguna prueba de permisos detecta, precisamente porque el rol era el correcto; la lección es que controlar el rol y controlar el alcance son dos cosas distintas.

La corrección fue atar cada ficha de médico a su cuenta (`medicos.usuario_id`, migración **V12**) y deducir la agenda del propio JWT. El detalle que más vueltas costó fue decidir qué responder cuando un profesional pide la agenda de otro: lo natural parece un 403, pero un 403 confirma que ese médico existe, y los identificadores son correlativos. Así que se le devuelve la suya con un 200 corriente y el parámetro se ignora en silencio. Un `ADMIN` sí puede elegir, pero está obligado a indicar cuál: el antiguo "médico 1 por defecto" era una suposición disfrazada de valor por defecto.

Frenar la fuerza bruta sin romper al paciente. El límite de peticiones no es solo una defensa contra el robo de contraseñas. Cada intento de inicio de sesión cuesta un `BCrypt` de coste 12, elegido a propósito por ser lento, de modo que un bucle de intentos no solo prueba claves: se come la CPU del contenedor. El filtro es un cubo de fichas en memoria, deliberadamente simple porque la aplicación corre en un único contenedor, y cuenta por dirección y por ruta, de forma que agotar el cupo del formulario de contacto no deja a nadie sin poder reservar. Detrás del proxy inverso la dirección real se toma de `X-Forwarded-For`, no la de la red interna de Docker; si el puerto de la aplicación se publicara directamente, esa cabecera podría falsearse y el límite dejaría de valer, así que el fichero de despliegue lo advierte donde se tomaría esa decisión.

3.21 Despliegue

Uno de los objetivos era que el sistema completo se pudiera levantar sin instalar nada manualmente, ni siquiera por alguien sin experiencia previa. Por eso todo el despliegue se reduce a un único comando, que construye las imágenes y arranca los contenedores en el orden correcto:

```
docker compose up -d --build
```

Esto levanta tres servicios coordinados: `postgres` (la base de datos), `backend` en el puerto 8080 (la API Spring Boot) y `frontend` en el puerto 80 (nginx sirviendo la web y haciendo de proxy de `/api`). El `docker-compose` define dependencias y comprobaciones de salud, de modo que el backend no se da por listo hasta que la base de datos responde. El artefacto del backend es un **JAR de Spring Boot con servidor embebido** (generado por `spring-boot-maven-plugin`), el esquema de la base de datos se versiona con Flyway —que aplica las migraciones pendientes en el primer arranque— y la configuración se separa por perfiles (`application-postgres.yml` y `application-mysql.yml`), existiendo además un perfil opcional de MySQL con `-profile mysql`. La ventaja práctica es enorme: cualquier entorno queda idéntico al de desarrollo, eliminando el clásico “en mi máquina funciona”.

Dominio e identidad de marca. De cara al salto a producción se ha registrado el dominio `dragramonte.com`, una marca personal corta, profesional y fácil de dictar por teléfono (frente a alternativas más largas que se descartaron). La aplicación ya está preparada para ese dominio: los pies de página, la página de contacto, las etiquetas `Open Graph` y la política de CORS del backend apuntan a `dragramonte.com`, y el correo de contacto utiliza la propia marca (`contacto@dragramonte.com`). El despliegue, además, ya está **en producción**: la aplicación se ejecuta en un **servidor propio (VPS) con Docker**, empaquetada con un `Dockerfile` *multi-stage* en el que el backend Spring Boot sirve a la vez la API y el frontend en un **único contenedor** —junto a un PostgreSQL que solo escucha en la red interna—, con **Caddy** como proxy inverso delante, y es **accesible públicamente con HTTPS** (certificado Let’s Encrypt, renovado de forma automática) en `https://www.dragramonte.com`. Llegar a un dominio propio, con certificado válido y la aplicación abierta a cualquier navegador, es una madurez poco habitual en un proyecto de este tamaño; la migración de identidad de marca ya estaba hecha en el código y ahora se acompaña del despliegue real.

Demo pública. Junto al despliegue real existe una **demo pública** en `https://maragramonte.github.io/dr-agramonte/`, pensada para que cualquiera pueda

probar la aplicación entera sin credenciales, sin instalar nada y sin tocar los datos reales de la consulta. No es una maqueta aparte: son las mismas páginas y el mismo cliente de API que el frontend de producción, acompañados de una **API simulada que se ejecuta en el propio navegador** y que intercepta las llamadas a `/api/...` devolviendo los mismos contratos JSON que los DTOs del backend. Reproduce también sus reglas de negocio: el reparto de la agenda por centro de la migración V6, el 409 cuando la franja ya está ocupada y el 401/403 según el rol del usuario. Los datos son ficticios y viven en el `localStorage` del visitante, de modo que cada persona trabaja sobre su propia copia y nada sale de su navegador; el canal de Telegram se representa con un chat simulado dentro de la página, con los mismos textos que genera `CitaNotificationService`, sin llegar a usar el bot real.

La demo se genera con `scripts/build-demo.sh`, que copia el frontend e inyecta la API simulada **sin modificar** el directorio `frontend/`: la imagen de producción nunca incluye el modo demo. Un flujo de GitHub Actions la construye en cada cambio que la afecte —también en los *pull request*, donde solo se construye y no se publica— y la despliega en GitHub Pages, y todas sus páginas se marcan `noindex` para no competir en buscadores con el dominio real. Conviene dejar escrito su límite: al no haber base de datos detrás, la demo *no* puede demostrar la concurrencia real —el bloqueo pesimista solo existe contra PostgreSQL—, que es justamente lo que cubren las pruebas de integración.

3.22 Sostenibilidad y eficiencia (green computing)

La sostenibilidad de este proyecto no es una etiqueta añadida al final, sino una consecuencia directa de las decisiones de diseño. Un software es sostenible cuando consume los recursos justos —energía, ancho de banda, hardware y, en última instancia, dinero— para hacer su trabajo, y cuando además *sustituye* procesos físicos más costosos. Dr. Agramonte actúa en los dos frentes: reduce el gasto material de la consulta y se ha construido para gastar poco al ejecutarse.

Digitalización y ahorro de papel. El primer impacto es el más evidente: la aplicación elimina la agenda de papel y los apuntes manuales que hoy usan muchas consultas pequeñas. Las citas, sus cambios y su historial viven en la base de datos, y los avisos llegan por canales digitales (SMS o WhatsApp vía Twilio) en lugar de recordatorios impresos o llamadas que consumen tiempo y, a veces, desplazamientos. Menos papel significa menos impresión, menos almacenamiento físico y menos residuos; y al evitar visitas o llamadas innecesarias para gestionar una cita, se reduce también el desplazamiento del paciente, con

su correspondiente huella. Es la traducción concreta del objetivo de desarrollo sostenible **ODS 12** (producción y consumo responsables) a una consulta médica.

Software eficiente, menos energía. La eficiencia técnica es, en la práctica, eficiencia energética: cada ciclo de CPU y cada byte transferido que se ahorran se traducen en menos consumo, tanto en el servidor como en los miles de dispositivos de los pacientes. Varias decisiones del proyecto van en esa dirección:

- **Frontend sin framework.** Prescindir de React o Vue evita arrastrar un `node_modules` de cientos de megas y enviar grandes paquetes de JavaScript al navegador. La web carga más rápido y exige menos CPU al cliente, lo que en un parque grande de dispositivos no es despreciable.
- **Recursos vendorizados.** Servir Chart.js desde el propio dominio en lugar de una CDN externa reduce peticiones de red a terceros y dependencias innecesarias.
- **Cálculo en la base de datos.** El cuadro de mando agrega los datos con consultas `GROUP BY` en PostgreSQL en lugar de traer todas las filas a memoria y procesarlas en la aplicación. Mover el cálculo a donde están los datos transfiere menos información y gasta menos memoria y CPU en el backend.
- **Una sola base de datos relacional.** No hay un clúster de servicios (Redis, Mongo) corriendo y consumiendo energía en reposo: un único PostgreSQL cubre las necesidades reales del volumen de una consulta.
- **Autenticación *stateless*.** El token JWT evita guardar sesiones en el servidor, reduciendo el estado que hay que almacenar y mantener.

Infraestructura ligera y escalable a la baja. El backend se empaqueta con un `Dockerfile` *multi-stage*, de modo que la imagen final solo contiene el JRE y el JAR, sin las herramientas de compilación: imágenes más pequeñas significan menos disco, descargas más rápidas y menos energía en cada despliegue. El sistema completo cabe holgadamente en un servidor modesto (un VPS pequeño basta para una consulta), y los contenedores pueden apagarse cuando no se usan. Frente a la tendencia de sobredimensionar (*just in case*), aquí se dimensiona para la necesidad real (*just enough*), que es la forma más directa de **green computing**: no encender lo que no hace falta.

Sostenibilidad del propio software. Un sistema también es sostenible si dura. La reproducibilidad con Docker y Flyway evita reconstrucciones manuales propensas a error; el análisis de vulnerabilidades con OWASP `dependency-check` mantiene las dependencias sanas en el tiempo; y un código legible, sin capas innecesarias, es más fácil de mantener durante años por otra persona. Un software que se mantiene vivo y seguro no hay que reescribirlo desde cero cada poco, lo que es una forma de no malgastar esfuerzo ni recursos.

Limitación honesta. No se ha realizado una medición formal de la huella de carbono ni del consumo energético real (con herramientas como auditorías de *Lighthouse* o medidores de consumo del contenedor); las afirmaciones anteriores son cualitativas y se apoyan en las decisiones de arquitectura, no en métricas instrumentadas. Cuantificar ese ahorro —por ejemplo, comparar el consumo del frontend frente a una versión con framework— es una línea de mejora natural y coherente con el resto del proyecto.

3.23 Planificación temporal

El desarrollo se organizó en fases encadenadas a lo largo de cuatro meses, partiendo del análisis y el diseño de datos antes de escribir el grueso del backend, y dejando la redacción de la memoria como una tarea transversal que acompañó a todo el proyecto. El diagrama de Gantt de la figura siguiente resume esa planificación por hitos; en la práctica algunas fases se solaparon (por ejemplo, las pruebas se fueron escribiendo a la vez que avanzaban backend y frontend), algo habitual y deseable en un desarrollo iterativo.

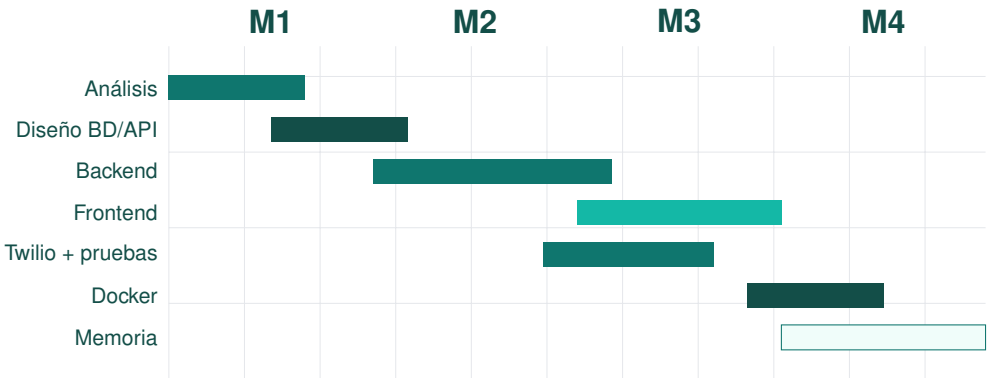


Figura 3.6: Diagrama de Gantt por hitos (M1–M4).

3.24 Pruebas y validación

La estrategia de pruebas se centró en los **caminos críticos**, aquellos en los que un fallo tendría consecuencias reales: la no duplicación de reservas, el respeto de los permisos por rol y la validez del token de autenticación. La filosofía no fue "probarlo todo" de forma superficial, sino probar bien lo que importa, combinando tres niveles —unitario, de seguridad e integración— para cubrir el comportamiento desde distintos ángulos. El proyecto incorpora una batería de **102 pruebas automatizadas**, ejecutable con `mvn test` desde `backend/`:

- **Unitarias (caja blanca):** `CitaServiceTest` (reserva sobre franja libre confirma y bloquea el horario; franja ocupada → 409 sin persistir; cancelación por no propietario → 403; re-cancelación → 409). `JwtProviderTest` (generación, validación y expiración del token firmado). `TelegramVinculacionServiceTest` (el token de vinculación se emite con caducidad, dos llamadas seguidas no repiten token, uno caducado no cuenta como vinculación pendiente, la desvinculación es idempotente y con el canal apagado la petición responde 503).
- **Seguridad / RBAC (caja negra, WebMvc):** `CitaControllerWebMvcTest` importa la `SecurityConfig` real: rol MEDICO sobre rutas de paciente → 403; acceso anónimo bloqueado; rol PACIENTE → 201. `TelegramVinculacionControllerWe` comprueba que las tres operaciones del canal exigen sesión y que siempre actúan sobre el usuario del *principal*, nunca sobre uno indicado por parámetro. `TelegramWebhookControllerWebMvcTest` cubre la ruta pública del *webhook*: sin la cabecera del secreto o con un secreto que no coincide → 401 sin tocar la base de datos; con el secreto correcto, `/start <token>` vincula el chat; y un token caducado no vincula.
- **Integración end-to-end (Testcontainers + PostgreSQL real):** `ReservaPublicaIntegratio` *(a)* **concurrencia real**: dos transacciones simultáneas, en hilos distintos sincronizados con una `CyclicBarrier`, compiten por la misma franja; exactamente una confirma (201) y la otra recibe 409, demostrando el bloqueo pesimista sobre PostgreSQL —algo que un H2 en memoria no reproduce y que un *mock* no prueba—; *(b)* segunda reserva secuencial sobre la franja ya ocupada → 409; *(c)* reserva pública persistida y recuperable *solo* con su token de acceso, no con el email; *(d)* agenda por centro: el hueco se filtra por ubicación y la cita hereda el centro del horario.
- **Activación de una cuenta invitada (Testcontainers + GreenMail):** `ActivacionCuentaInte` recorre el camino completo contra un PostgreSQL real y un servidor SMTP

real en memoria: se reserva como invitado, se intenta el alta con ese mismo correo, se **lee el mensaje recibido** para extraer el token del enlace, se activa la cuenta y se comprueba después que se puede iniciar sesión, que las citas de cuando era invitado siguen ahí y que el enlace no sirve una segunda vez. El correo se envía y se lee de verdad porque es la pieza de seguridad: un envío simulado comprobaría que se invoca un método, no que el mecanismo funciona. Lo acompañan las unitarias de `ActivacionCuentaServiceTest` y `AuthServiceTest`, que fijan que pedir la activación no modifica ni la contraseña ni los datos de la cuenta, y que un token inexistente, uno caducado y uno ya usado devuelven la misma respuesta.

- **Aislamiento entre agendas:** en `@WebMvcTest` se comprueba que un médico que pide la agenda de otro recibe la suya, con un 200 corriente, y que el servicio nunca llega a consultarse con el identificador ajeno; y en integración, con dos médicos reales dados de alta con su cuenta, que ninguna de las dos respuestas contiene un solo dato del otro.
- **Generación de la agenda:** `GeneradorAgendaSchedulerTest` fija el horario de consulta, el cierre de los domingos y el reparto por centro, y que una franja ya existente no se duplica ni se reabre. La prueba de integración reproduce el día noventa y uno: vacía la agenda futura, ejecuta el generador y vuelve a reservar.
- **Límite de peticiones:** `RateLimitFilterTest` comprueba el 429 con `Retry-After`, que el cupo es por dirección y por ruta, que detrás del proxy cuenta la dirección real y que las lecturas nunca se cortan.
- **Análisis de vulnerabilidades:** `OWASP dependency-check` (`mvn org.owasp:dependency-che`
- **Manuales:** compilación Maven, arranque Docker Compose y health check; flujo registro/login → reservar → mis citas → cancelar; parámetro `?servicio=`; validación de formularios (email, teléfono español).

Tabla 3.4: Matriz de trazabilidad requisito ↔ prueba (RTM)

Requisito	Prueba	Tipo
No doble reserva (2 hilos a la vez)	`ReservaPublicaIntegrationTest`	Integración
RBAC / menor privilegio	`CitaControllerWebMvcTest`	Seguridad
JWT firmado válido	`JwtProviderTest`	Unitaria
Solo el dueño cancela	`CitaServiceTest`	Unitaria
No re-cancelar	`CitaServiceTest`	Unitaria
Webhook público sin secreto válido	`TelegramWebhookControllerWebMvcTest`	Seguridad
Canal Telegram solo del propio usuario	`TelegramVinculacionControllerWebMvcTest`	Seguridad
Token de vinculación caducado	`TelegramVinculacionServiceTest`	Unitaria
El registro no toma la cuenta invitada	`AuthServiceTest`	Unitaria
Activación de principio a fin (SMTP real)	`ActivacionCuentaIntegrationTest`	Integración
Un médico no ve la agenda de otro	`CitaControllerWebMvcTest`	Seguridad
La agenda no se agota	`GeneradorAgendaSchedulerTest`	Unitaria
Fuerza bruta cortada	`RateLimitFilterTest`	Seguridad
No se reserva en el pasado	`CitaServiceTest`	Unitaria

Requisito de las pruebas de integración. Las diecisiete pruebas de integración levantan un PostgreSQL real mediante Testcontainers y, por tanto, necesitan un motor Docker en marcha. Cuando no lo hay, la propia suite las marca como *saltadas* en lugar de fallar, de modo que `mvn test` sigue siendo verde en una máquina sin Docker; para una verificación completa conviene arrancarlo y ejecutar las 102. La integración continua sí exige que se ejecuten todas: el *workflow* revisa los informes de Surefire y falla si alguna quedó sin ejecutar, porque un verde que esconde pruebas saltadas es peor que un rojo.

Mejoras futuras de calidad. Informe de cobertura formal con **JaCoCo** ($\geq$ 80 % medido), pruebas E2E (Playwright/Cypress), auditoría **Lighthouse** documentada y escenario de alta disponibilidad (réplicas/failover), hoy contemplado como diseño y no ejecutado en esta infraestructura.

Capítulo 4:

Conclusiones

Se ha entregado una aplicación funcional y desplegable que responde al problema de gestión de citas, con arquitectura clara, código mantenible y **pruebas automatizadas** que cubren los caminos críticos (409 por doble reserva, 403 por rol, JWT firmado, cancelación). Los objetivos técnicos del repositorio se cumplen en PostgreSQL, Spring Boot, frontend PWA y los dos canales de aviso (Twilio y Telegram) integrados en Java.

Cumplimiento de los objetivos. Retomando los ocho objetivos específicos planteados al inicio, el balance es el siguiente:

1. *Modelo de datos en PostgreSQL con Flyway.* **Cumplido:** seis tablas, migraciones V1–V13 y perfiles `postgres/mysql`.
2. *API REST con Spring Boot, JWT y roles.* **Cumplido:** controladores por recurso, autenticación JWT y autorización por rol en `SecurityConfig`.
3. *Integridad de reservas con `SELECT FOR UPDATE`.* **Cumplido y verificado:** la prueba de integración confirma que, ante dos transacciones simultáneas en hilos distintos sobre la misma franja, exactamente una reserva prospera (201) y la otra recibe 409.
4. *Frontend responsive, PWA y reserva accesible.* **Cumplido:** reserva por pasos, modo oscuro, selección de centro y service worker.
5. *Notificaciones con recordatorio programado.* **Cumplido y ampliado:** avisos a médico y paciente y recordatorio a $\sim$24 h vía `@Scheduled`, por **dos canales independientes** —Twilio (SMS/WhatsApp) y un bot de Telegram con vinculación por token de un solo uso—, de forma que la caída o desactivación de uno no deja al paciente sin aviso.
6. *Despliegue reproducible con Docker Compose.* **Cumplido:** un único comando levanta base de datos, backend y frontend; y, más allá del objetivo,

la aplicación está **desplegada en un servidor propio** (VPS con Docker y Caddy) y es pública con HTTPS en `https://www.dragramonte.com`, con una **demo pública** en GitHub Pages que permite probarla sin credenciales ni servidor.

7. *Validación de la calidad.* **Cumplido parcialmente:** pruebas unitarias, de seguridad e integración más `OWASP dependency-check`; queda pendiente medir la cobertura con JaCoCo y añadir pruebas E2E.

8. *Cuadro de mando de gestión (SGE).* **Cumplido:** endpoint `/api/estadisticas` con agregaciones `GROUP BY`, restringido por rol y representado con Chart.js (carga por médico, demanda por especialidad, reparto por centro y estado, evolución temporal y tasa de cancelación).

Limitaciones honestas: la cobertura no está medida con JaCoCo (se valida cualitativamente por ramas críticas), el panel médico no expone API de historial, no hay refresh token ni escenario de réplicas/failover. Son líneas de evolución natural del producto, no carencias de lo entregado.

Líneas de trabajo futuras. El proyecto está pensado para seguir creciendo más allá de la entrega. Las evoluciones más coherentes con lo ya construido son:

- **Panel clínico multi-centro:** con la agenda por centro ya implementada (horarios y citas asociados a su centro, disponibilidad filtrada), el siguiente paso sería un panel médico visual que conmute entre ubicaciones y muestre la ocupación de cada centro de un vistazo.
- **Recordatorios con confirmación previa:** permitir que el paciente confirme o cancele desde el propio aviso, reduciendo ausencias.
- **Historial de cambios de la cita:** que el paciente vea reprogramaciones y cancelaciones.
- **Página de ayuda / preguntas frecuentes:** apoyo específico para pacientes mayores o poco digitales.
- **Calidad:** informe de cobertura con JaCoCo, pruebas E2E (Playwright) y auditoría de accesibilidad documentada.
- **Cobro en línea:** integración de una pasarela de pago (por ejemplo Stripe con *captura manual*: se autoriza el importe al reservar y solo se cobra si la consulta se realiza, de modo que una ausencia no genera cargo), partiendo de la cobertura y la preferencia de pago que ya se capturan en la reserva.

- **Seguridad y escalado:** *refresh token*, réplicas de base de datos con *failover* y una infraestructura con mayor garantía de disponibilidad (hoy es un único servidor: si cae, cae la web).
- **Historial clínico:** API y panel clínico reales, con almacenamiento documental flexible por especialidad (p. ej. MongoDB), avanzando hacia una persistencia políglota.

Aprendizajes: lo que más ha enseñado este proyecto no es ninguna tecnología concreta, sino el criterio para decidir entre varias. Cuándo un bloqueo pesimista compensa frente a uno optimista, por qué un aviso debe salir después del *commit* y no durante, cuándo *no* añadir un framework. También que un sistema pequeño y verificable vale más que uno ambicioso a medias: cada afirmación de este documento se sostiene sobre código que se puede ejecutar.

Reflexión: la coherencia entre lo que un documento afirma y lo que el código hace es tan importante como la propia funcionalidad. Este documento describe la versión real del sistema, no una versión idealizada.

Agradecimientos

Este proyecto no es solo horas frente a una pantalla: es el reflejo de conversaciones, experiencias y realidades observadas durante años.

A mi **familia**, por el apoyo y la paciencia durante todo el desarrollo, y por ser quienes me enseñaron de cerca lo que significa cuidar de los demás.

A mis **amigos y conocidos del entorno sanitario**, que compartieron sus experiencias y sus días difíciles y, sin pretenderlo, inspiraron el problema que este proyecto intenta aliviar. Esta herramienta es, en el fondo, para ellos.

A mis **profesores del ciclo**, por darme las herramientas necesarias —técnicas y de criterio— para convertir una idea observada en la realidad en una aplicación funcional, y por ayudarme a entender que un proyecto pequeño y sólido vale más que una idea enorme a medias.

Y a mis **compañeras y compañeros de DAM**, que probaron la aplicación sin filtros y dieron un *feedback* honesto que mejoró el resultado.

Bibliografía

Bibliografía

[1] Spring. (2024). *Spring Boot 3.5 reference documentation*. https://spring.io/projects/spring-boot

[2] Spring. (2024). *Spring Security reference*. https://docs.spring.io/spring-security/reference/

[3] Parlamento Europeo y Consejo de la Unión Europea. (2016). *Reglamento (UE) 2016/679, de 27 de abril de 2016, relativo a la protección de las personas físicas (RGPD)*. Diario Oficial de la Unión Europea. https://eur-lex.europa.eu/eli/reg/2016/679/oj

[4] Jefatura del Estado. (2002). *Ley 41/2002, de 14 de noviembre, básica reguladora de la autonomía del paciente y de derechos y obligaciones en materia de información y documentación clínica*. BOE núm. 274. https://www.boe.es/eli/es/l/2002/11/14/41

[5] World Wide Web Consortium. (2021). *Web Content Accessibility Guidelines (WCAG) 2.1*. https://www.w3.org/WAI/WCAG21/quickref/

[6] Docker, Inc. (s.f.). *Docker Compose documentation*. Recuperado en junio de 2026, de https://docs.docker.com/compose/

[7] Twilio. (s.f.). *Messaging API documentation*. Recuperado en junio de 2026, de https://www.twilio.com/docs/messaging

[8] Telegram. (s.f.). *Telegram Bot API*. Recuperado en agosto de 2026, de https://core.telegram.org/bots/api

[9] Redgate. (s.f.). *Flyway documentation*. Recuperado en junio de 2026, de https://documentation.red-gate.com/fd

[10] Testcontainers. (s.f.). *Testcontainers for Java*. Recuperado en junio de 2026, de https://java.testcontainers.org/

[11] Doctoralia. (s.f.). *Plataforma para profesionales*. Recuperado en junio de 2026, de https://www.doctoralia.es

Anexo

Manual de usuario (resumen)

1. **Registro / acceso (opcional):** desde el propio menú de navegación, crear cuenta (rol `PACIENTE`) o iniciar sesión. El sistema devuelve un token JWT que el navegador guarda para las peticiones protegidas. También se puede reservar sin registrarse.
2. **Elegir servicio (opcional):** en `servicios.html`, pulsar un servicio abre la reserva con el tipo de consulta preseleccionado (`reservar.html?servicio=...`).
3. **Reservar:** en `reservar.html` seguir los pasos centro $\rightarrow$ calendario $\rightarrow$ hora $\rightarrow$ datos $\rightarrow$ confirmación, aceptando la política de privacidad.
4. **Mis citas:** consultar las citas propias y su estado. El paciente recibe la confirmación y el recordatorio ($\sim$24 h antes) por SMS o WhatsApp si tiene teléfono, y por Telegram si ha vinculado el bot desde esa misma pantalla.
5. **Cancelar:** desde “Mis citas”, cancelar libera la franja y notifica al médico.
6. **Tema claro/oscuro:** el botón luna/sol alterna el modo y se recuerda la preferencia.